

SERWISY I KOMENDY

SPIS TREŚCI

Spis treści	1
Cel zajęć.....	1
Rozpoczęcie	1
Uwaga	1
Serwis WeatherUtil	1
Komendy	4
Commit projektu do GIT.....	7
Podsumowanie.....	8

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- zamykanie reużywalnej logiki biznesowej w serwisach;
- wykorzystanie serwisów w kontrolerach;
- wykorzystanie serwisów w komendach;
- tworzenie komend konsolowych.

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie zasad tworzenia serwisów, komend.

Wejściówka?

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do Plik -> Informacje -> Właściwości -> Właściwości zaawansowane -> Niestandardowe i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub Ctrl+A -> F9.

SERWIS WEATHERUTIL

Utwórz nową klasę `src/Service/WeatherUtil.php`, a w niej deklaracje dwóch metod:

- `getWeatherForLocation($location)` – odpowiedzialna za pobranie pomiarów (prognoza pogody) na podstawie encji lokalizacji;
- `getWeatherForCountryAndCity($countryCode, $cityName)` – odpowiedzialna za pobranie pomiarów na podstawie kodu kraju i nazwy miasta. Wewnętrzna implementacja sprowadza się do pobrania lokalizacji na podstawie kodu kraju i nazwy miasta, a następnie wywołania metody `getWeatherForLocation` dla otrzymanej lokalizacji.

```
<?php
declare(strict_types=1);

namespace App\Service;

use App\Entity\Location;
use App\Entity\Measurement;

class WeatherUtil
{
    /**
     * @return Measurement[]
     */
    public function getWeatherForLocation(Location $location): array
    {
        return [];
    }

    /**
     * @return Measurement[]
     */
    public function getWeatherForCountryAndCity(string $countryCode, string $city): array
    {
        return [];
    }
}
```

Zmodyfikuj `WeatherController`, żeby wykorzystywał nowy serwis:

<pre>class WeatherController extends AbstractCon { #[Route('/weather/{country}/{city}', na public function city(#[MapEntity(mapping: ['country' => Location \$location, MeasurementRepository \$repository,): Response { \$measurements = \$repository->findBy } }</pre>	<pre>12 13 13 14 14 15 15 16 16 17 17 18 18 19 19 20 20 21 21 22 22 23 23 24 24 25 25 26 26 27 27 28 28 29</pre>	<pre>class WeatherController extends AbstractController { #[Route('/weather/{country}/{city}', name: 'app_weather', requ public function city(#[MapEntity(mapping: ['country' => 'country', 'city' => 'c Location \$location, WeatherUtil \$util,): Response { \$measurements = \$util->getWeatherForLocation(\$location); } }</pre>
--	--	---

Sprawdź, czy strona prognozy pogody wciąż działa. Nie powinno być żadnych błędów, jednakże zwracana lista pomiarów będzie pusta.

Zaimplementuj ciało metody `getWeatherForLocation()`. Wstaw zrzut ekranu kodu:

```

/**
 * @return Measurement[]
 */
public function getWeatherForLocation(Location $location): array
{
    return $this->measurementRepository->findBy([
        'location' => $location
    ]);
}

```

Punkty:

0

1

Zaimplementuj ciało metody `getWeatherForCountryAndCity()`. Wstaw zrzut ekranu kodu:

```

/**
 * @return Measurement[]
 */
public function getWeatherForCountryAndCity(string $countryCode, string $city): array
{
    $location = $this->locationRepository->findOneBy([
        'country' => $countryCode,
        'city' => $city,
    ]);

    $measurements = $this->getWeatherForLocation($location);

    return $measurements;
}

```

Punkty:

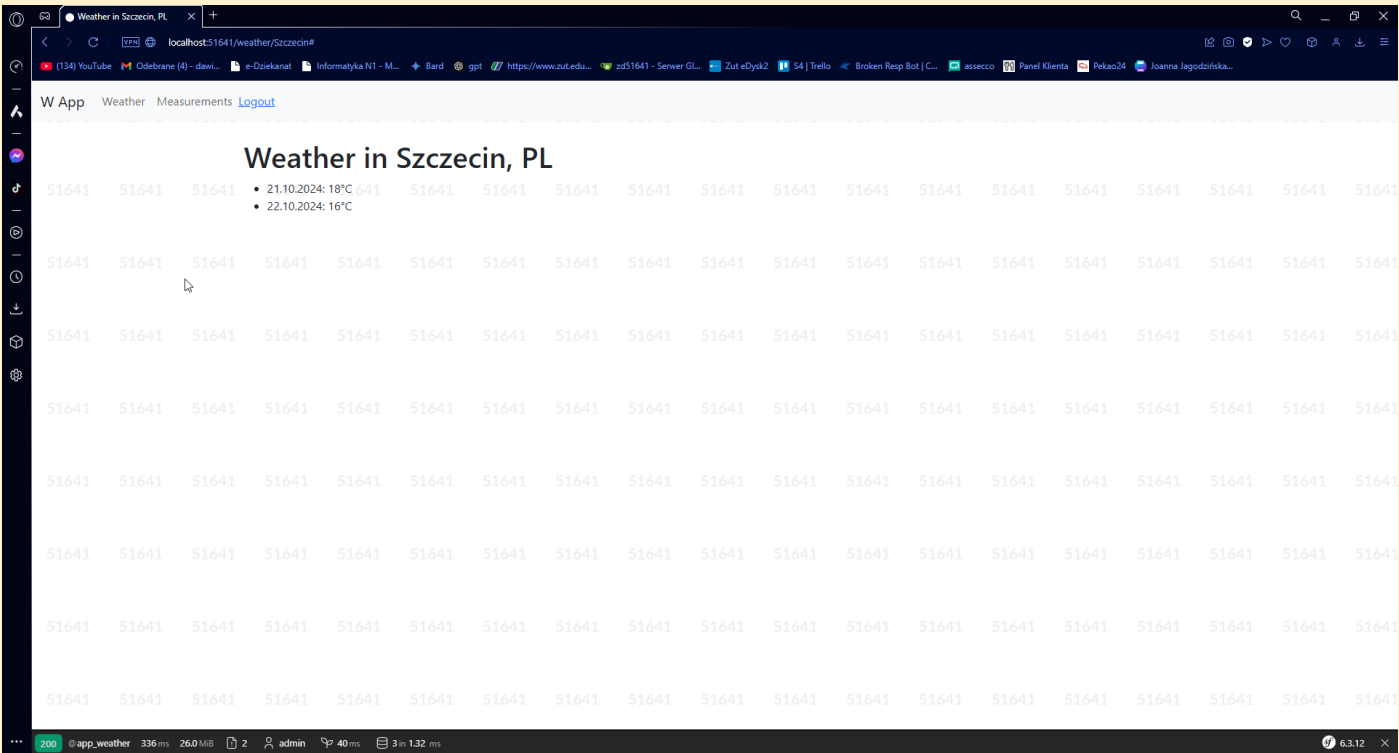
0

1

Wstaw zrzut ekranu kodu kontrolera wykorzystującego metodę z serwisu:

```
class WeatherController extends AbstractController
{
    #[Route('/weather/{location}', name: 'app_weather', requirements: ['location' => '[a-zA-Z]+'])]
    public function city(string $location, LocationRepository $locationRepository, WeatherUtil $util): Response
    {
        $locationEntity = $locationRepository->findOneBy(['city' => $location]);
        $measurements = $util->getWeatherForLocation($locationEntity);
        return $this->render('weather/city.html.twig', [
            'location' => $locationEntity,
            'measurements' => $measurements,
        ]);
    }
}
```

Wstaw zrzut ekranu prognozy pogody z pomiarami pobranymi z serwisu:



Punkty:	0	1
---------	---	---

KOMENDY

Wykorzystaj komendę `make:command` do utworzenia komendy `weather:location`, służącej do pobierania prognozy pogody dla lokalizacji:

```
php .\bin\console make:command

Choose a command name (e.g. app:agreeable-pizza):
> weather:location

created: src/Command/WeatherLocationCommand.php
```

Success!

Next: open your new command class and customize it!

Find the documentation at <https://symfony.com/doc/current/console.html>

Edytuj utworzony plik `src/Command/WeatherLocationCommand.php`:

- podłącz serwis do konstruktora;
- ustaw wymagany argument id lokalizacji;
- w `execute()` wykorzystaj serwis do pobrania prognozy pogody;
- wydrukuj prognozę pogody na widok.

Przykładowe wywołanie:

```
php .\bin\console weather:location 1
Location: Szczecin
        2023-09-26: 18
        2023-09-27: 17
```

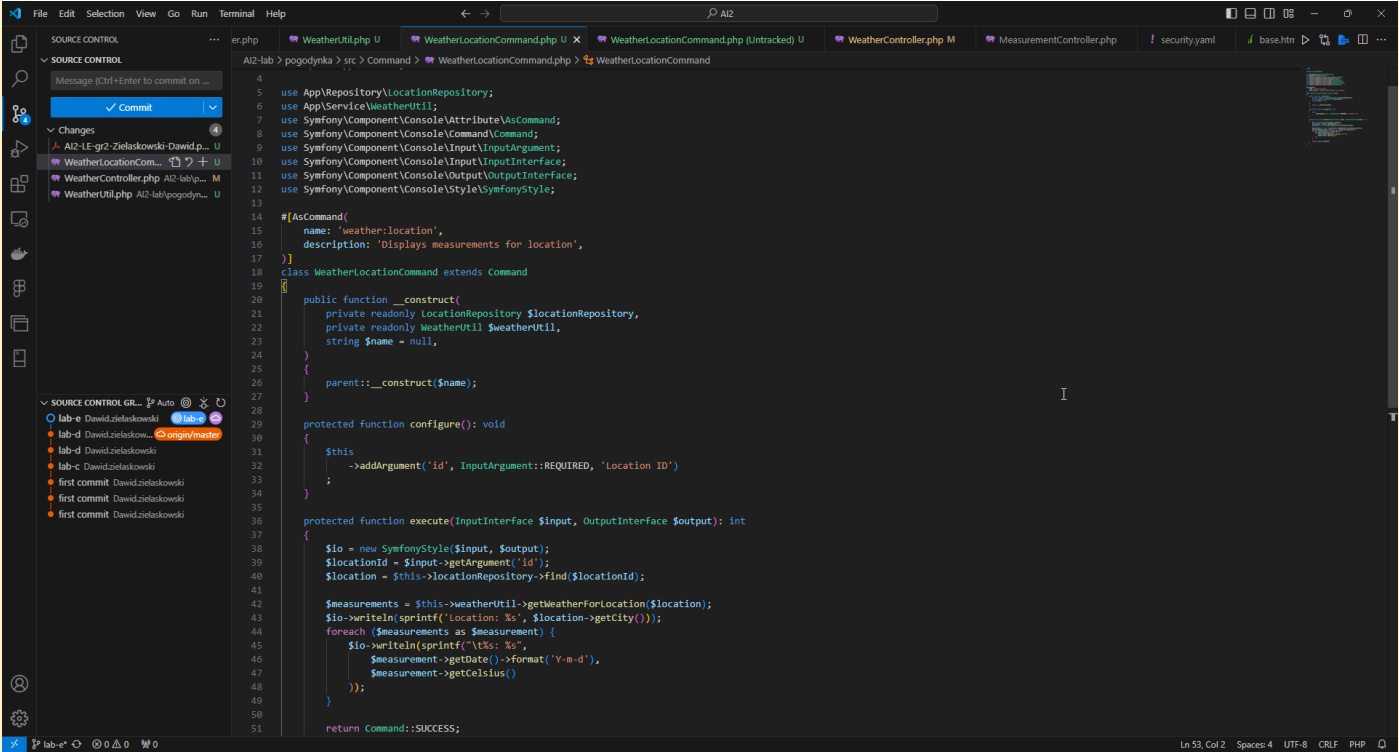
Przykładowy kod:

```
protected function execute(InputInterface $input, OutputInterface $output): int
{
    $io = new SymfonyStyle($input, $output);
    $locationId = $input->getArgument('id');
    $location = $this->locationRepository->find($locationId);

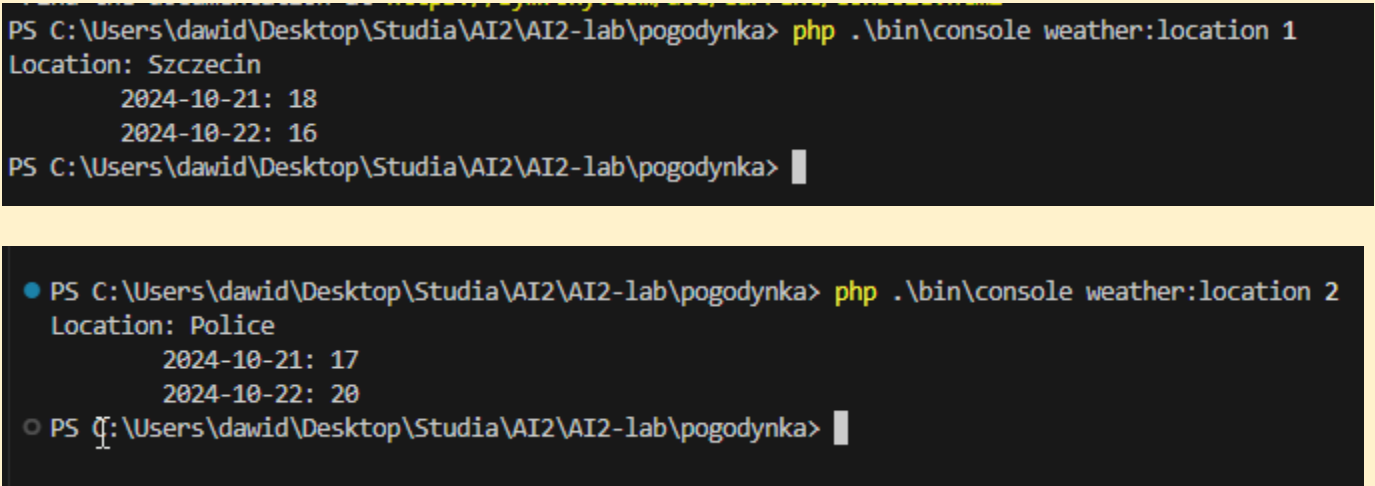
    $measurements = $this->weatherUtil->getWeatherForLocation($location);
    $io->writeln(sprintf('Location: %s', $location->getCity()));
    foreach ($measurements as $measurement) {
        $io->writeln(sprintf("\t%s: %s",
            $measurement->getDate()->format('Y-m-d'),
            $measurement->getCelsius()
        ));
    }

    return Command::SUCCESS;
}
```

Wstaw zrzuty ekranu kodu całości komendy:



Wstaw zrzuty ekranu wyniku działania komendy dla dwóch lokalizacji:

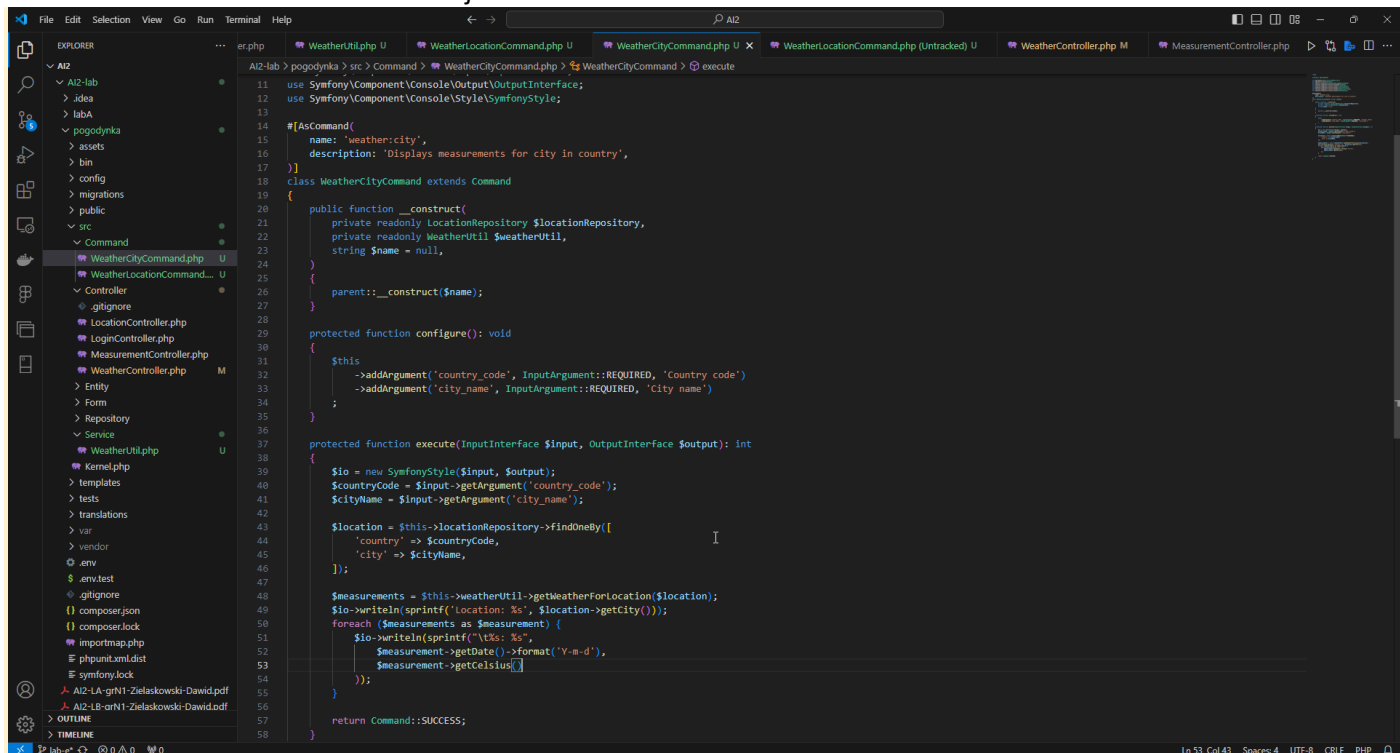


Punkty:	0	1
---------	---	---

W analogiczny sposób utwórz komendę do pobierania lokalizacji na podstawie kodu kraju i nazwy miejscowości.

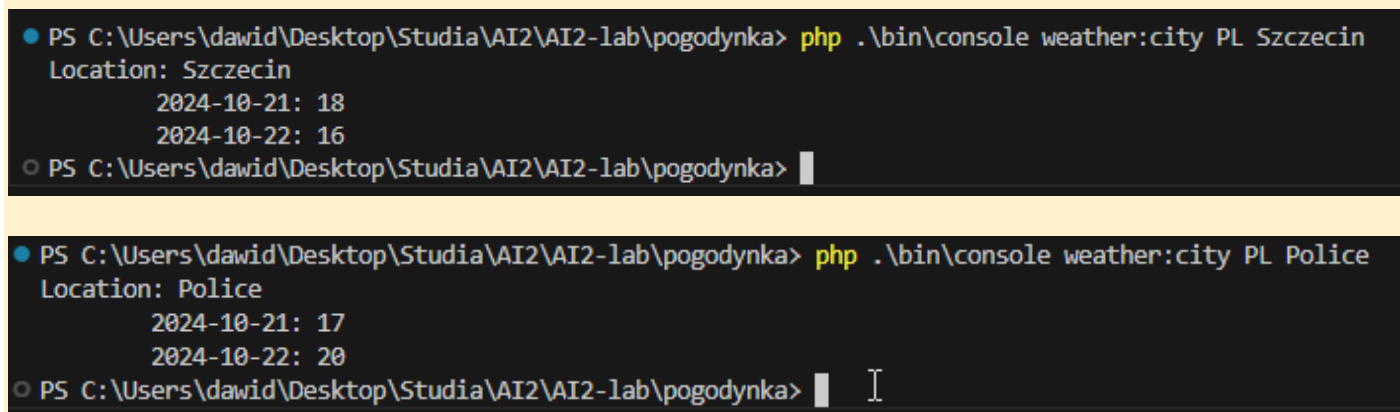
Wstaw zrzuty ekranu kodu całości komendy:

AI2 LAB F – Zielaskowski Dawid – Wersja 1



```
er.php  WeatherUtil.php U  WeatherLocationCommand.php U  WeatherCityCommand.php U  WeatherLocationCommand.php (Untracked) U  WeatherController.php M  MeasurementController.php U
AI2-lab > pogodynka > src > Command > WeatherCityCommand.php > execute
11 use Symfony\Component\Console\Output\OutputInterface;
12 use Symfony\Component\Console\Style\SymfonyStyle;
13
14 #[AsCommand(
15     name: 'weather:city',
16     description: 'Displays measurements for city in country',
17 )]
18 class WeatherCityCommand extends Command
19 {
20     public function __construct(
21         private readonly LocationRepository $locationRepository,
22         private readonly WeatherUtil $weatherUtil,
23         string $name = null,
24     ) {
25         parent::__construct($name);
26     }
27
28     protected function configure(): void
29     {
30         $this
31             ->addArgument('country_code', InputArgument::REQUIRED, 'Country code')
32             ->addArgument('city_name', InputArgument::REQUIRED, 'City name')
33         ;
34     }
35
36     protected function execute(InputInterface $input, OutputInterface $output): int
37     {
38         $io = new SymfonyStyle($input, $output);
39         $countryCode = $input->getArgument('country_code');
40         $cityName = $input->getArgument('city_name');
41
42         $location = $this->locationRepository->findOneBy([
43             'country' => $countryCode,
44             'city' => $cityName,
45         ]);
46
47         $measurements = $this->weatherUtil->getWeatherForLocation($location);
48         $io->writeln(sprintf('Location: %s', $location->getCity()));
49         foreach ($measurements as $measurement) {
50             $io->writeln(sprintf("\t%s: %s",
51                 $measurement->getDate()->format('Y-m-d'),
52                 $measurement->getCelsius()
53             ));
54         }
55
56         return Command::SUCCESS;
57     }
58 }
```

Wstaw zrzuty ekranu wyniku działania komendy dla dwóch miejscowości:



```
PS C:\Users\dawid\Desktop\Studia\AI2\AI2-lab\pogodynka> php .\bin\console weather:city PL Szczecin
Location: Szczecin
2024-10-21: 18
2024-10-22: 16
PS C:\Users\dawid\Desktop\Studia\AI2\AI2-lab\pogodynka>

PS C:\Users\dawid\Desktop\Studia\AI2\AI2-lab\pogodynka> php .\bin\console weather:city PL Police
Location: Police
2024-10-21: 17
2024-10-22: 20
PS C:\Users\dawid\Desktop\Studia\AI2\AI2-lab\pogodynka>
```

Punkty:	0	1
---------	---	---

COMMIT PROJEKTU DO GIT

Zacommituj zmiany. Wyślij zmiany do repozytorium (push). Upewnij się, czy wszystko dobrze się wysłało. Jeśli tak, to z poziomu przeglądarki utwórz branch o nazwie `lab-f` na podstawie głównej gałęzi kodu.

Podaj link do brancha `lab-f` w swoim repozytorium:

<https://github.com/dawid-zielaskowski/ai2-pogodynka/tree/lab-f>

...link, np. <https://github.com/ideaspot-pl/ai2-pogodynka-202310/tree/lab-f...>

PODSUMOWANIE

W kilku zdaniach podsumuj zdobyte podczas tego laboratorium umiejętności.

...podsumowanie...

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.